

GUÍA MAESTRA DE ESTUDIO PARA EL PARCIAL

Materia: Lenguajes de Programación y Transducción / Compiladores

Equipo: Dylan Torres · Juan Gomez · Javier Rosero

1. Teoría de Lenguajes

2. Arquitectura del Compilador

3. Estructuras de Datos

4. Léxico, Regex & Autómatas

5. ANTLR 4 & Visitor

6. Banco de Preguntas

1. Teoría de Lenguajes y Gramáticas Formales

Un lenguaje en ciencias de la computación es un conjunto de cadenas formadas por símbolos tomados de un alfabeto específico bajo reglas sintácticas precisas.

📌 Elementos Matemáticos Básicos

- Alfabeto (Σ): Conjunto finito no vacío de símbolos.
Ej: $\Sigma = \{0, 1\}$ (binario), $\Sigma = \{a, b, c\}$.
- Símbolo: Elemento atómico indivisible de Σ .
- Cadena / Palabra (w): Secuencia finita de símbolos de Σ .
Longitud $|w|$: Cantidad de símbolos ($|101| = 3$).
- Cadena Vacía (ϵ o λ): Cadena de longitud cero ($|\epsilon| = 0$).
- Lenguaje (L): Cualquier subconjunto de Σ^* ($L \subseteq \Sigma^*$).

⚙️ Operaciones Fundamentales

- Concatenación (uv): Si $u = ab$ y $v = cd \Rightarrow uv = abcd$.
- Potencia (w^n): Repetición n veces ($a^3 = aaa$, $w^0 = \epsilon$).
- Unión ($L_1 \cup L_2$): Cadenas en L_1 o en L_2 .
- Cerradura de Kleene (L^*): 0 o más repeticiones de L . *¡Siempre contiene a ϵ !*
- Cerradura Positiva (L^+): 1 o más repeticiones ($L^+ = L L^*$).

Jerarquía de Chomsky (Clasificación de Gramáticas y Autómatas)

Noam Chomsky clasificó todas las gramáticas y lenguajes formales en 4 niveles concéntricos según el poder de sus reglas de producción:

Tipo	Nombre del Lenguaje	Forma de las Producciones	Autómata que lo Reconoce	Aplicación en Compiladores
Tipo 3	Lenguajes Regulares	$A \rightarrow aB$ o $A \rightarrow a$	Autómata Finito (AFD / AFND)	Análisis Léxico (Tokens, Identificadores, Regex).
Tipo 2	Libres de Contexto (GLC)	$A \rightarrow \alpha$ (A es 1 no terminal)	Autómata de Pila (Pushdown)	Análisis Sintáctico (Gramáticas, Parsers, ANTLR).
Tipo 1	Sensibles al Contexto	$\alpha A \beta \rightarrow \alpha \gamma \beta$ ($ \gamma \geq A $)	Autómata Linealmente Acotado	Análisis Semántico (Chequeo de tipos).
Tipo 0	Recursivamente Enumerables	$\alpha \rightarrow \beta$ (Sin restricciones)	Máquina de Turing	Computabilidad Universal general.

 Pregunta Fija de Parcial: ¿Por qué un Lexer (Regex) no puede contar paréntesis balanceados?

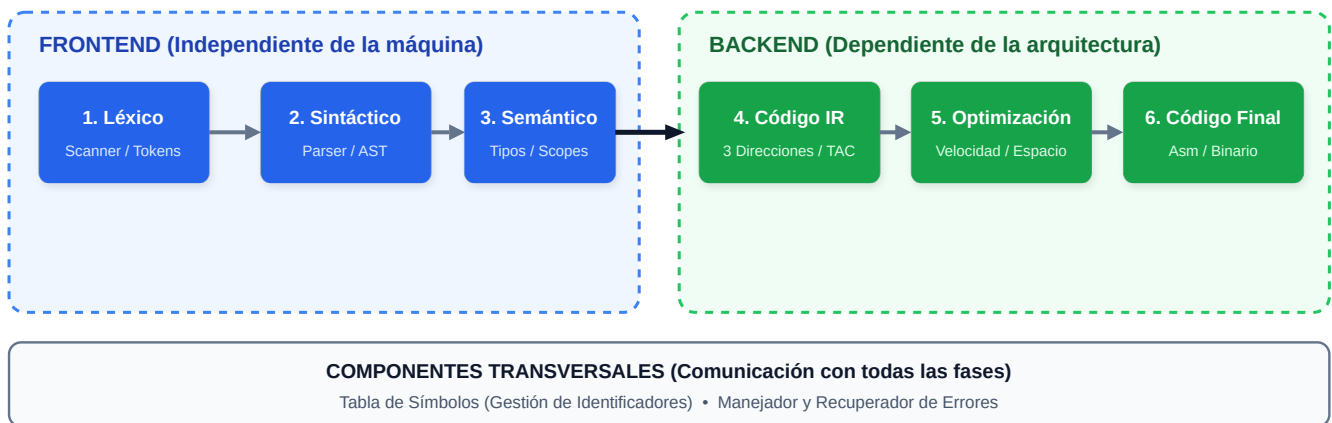
Un autómata finito (Tipo 3 / Regex) solo tiene memoria finita (sus estados). Para verificar estructuras anidadas arbitrariamente como `(((((...))))` o `anbn` se requiere memoria estructurada infinita (una Pila), lo cual pertenece a las Gramáticas Libres de Contexto (Tipo 2). Por eso el Lexer solo produce tokens y el Parser analiza la estructura jerárquica.

2. Arquitectura de Compiladores: Fases y Pipeline

💡 Analogía Intuitiva del Compilador

Un compilador es como un traductor profesional de libros: primero verifica que las letras formen palabras válidas en el diccionario (Léxico), luego revisa que las oraciones tengan sujeto y predicado correctos (Sintáctico), después comprueba que la historia tenga sentido lógico y coherencia de personajes (Semántico), traduce a un borrador universal (Código Intermedio), optimiza frases redundantes (Optimización) y finalmente imprime el libro en el formato final de la imprenta (Código Máquina).

El Pipeline Completo del Compilador



Descripción Detallada de cada Fase

- Análisis Léxico (Lexer):** Lee los caracteres del archivo de entrada y los agrupa en Tokens (tuplas como `<ID, "resultado">`, `<NUMBER, 42.5>`). Descarta comentarios y espacios en blanco.
- Análisis Sintáctico (Parser):** Comprueba que la secuencia de tokens cumpla la estructura jerárquica de la gramática. Construye el Árbol de Sintaxis Abstracta (AST).
- Análisis Semántico:** Valida el significado: comprobación de tipos (no sumar cadenas con booleanos), detección de variables no declaradas o duplicadas, y verificación de parámetros de funciones.
- Generación de Código Intermedio (IR):** Produce una representación intermedia independiente del hardware, típicamente en Código de Tres Direcciones (TAC) como `t1 = a + b`, `t2 = t1 * c`.
- Optimización de Código:** Aplica técnicas como: *Propagación de Constantes* (calcular `3 * 5` en compilación como `15`), *Eliminación de Código Muerto* (código inalcanzable), y *Reducción de Fuerza* (cambiar `x * 2` por `x + x` o desplazamiento de bits).
- Generación de Código Final:** Emite código ensamblador o binario para la arquitectura del procesador (x86_64, ARM) asignando registros de CPU de forma eficiente.

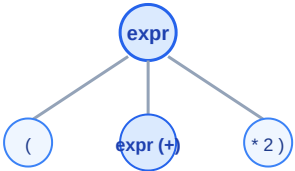
3. Estructuras de Datos en Compiladores

El rendimiento y la modularidad de un compilador dependen de 4 estructuras de datos fundamentales:

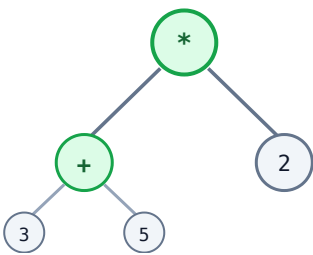
Estructura	Fase Principal	Propósito y Funcionamiento	Implementación en Memoria
Tabla de Símbolos	Todas las fases	Guarda los identificadores (variables, funciones, clases) junto con sus atributos: nombre, tipo, ámbito (scope), dirección de memoria y valor.	<code>Map<String, Symbol></code> o tablas hash encadenadas para soporte de ámbitos anidados.
Parse Tree (CST)	Sintáctico	Árbol sintáctico concreto que contiene absolutamente <i>todos</i> los elementos del texto fuente (incluyendo paréntesis y comas).	Estructura de árbol n-ario generada automáticamente por ANTLR.
AST (Abstract Syntax Tree)	Semántico e IR	Árbol sintáctico abstracto simplificado. Elimina la puntuación superflua y conserva solo operadores y operandos esenciales.	Nodos de clases polimórficas (<code>BinaryExpr</code> , <code>NumberLiteral</code>).
Pila de Parsing (Stack)	Parsers LL / LR	Gestiona el seguimiento del estado de la gramática, llamadas a funciones y evaluación de expresiones en notación postfija.	Estructura LIFO (<code>Stack<T></code> / <code>Deque<T></code>).

Comparación Visual: Parse Tree (CST) vs AST para `(3 + 5) * 2`

Parse Tree (CST - Concreto)



AST (Abstract Syntax Tree - Abstracto)



4. Análisis Léxico, Expresiones Regulares y Autómatas (AFD / AFND)

La base teórica del análisis léxico se fundamenta en la equivalencia matemática entre las Expresiones Regulares (ER), los Autómatas Finitos No Deterministas (AFND) y los Autómatas Finitos Deterministas (AFD).

1. Definición Formal de un Autómata Finito Determinista (AFD)

Un AFD es una 5-tupla: $M = (Q, \Sigma, \delta, q_0, F)$ donde:

- Q : Conjunto finito y no vacío de estados.
- Σ : Alfabeto de entrada.
- δ : Función de transición total $\delta: Q \times \Sigma \rightarrow Q$ (para cada estado y cada símbolo existe exactamente un estado destino).
- $q_0 \in Q$: Estado inicial único.
- $F \subseteq Q$: Conjunto de estados de aceptación o finales (representados con doble círculo).

2. Algoritmo de Thompson: De Expresión Regular a AFND

Permite construir sistemáticamente un AFND con transiciones ϵ para cualquier operador de una expresión regular:

Operación	Expresión Regular	Construcción de Thompson (Esquema del Autómata)
Símbolo Básico	<code>a</code>	<code>(inicio) — a —> ((final))</code>
Concatenación	<code>ab</code>	<code>(inicio) —[a]—> (medio) —[b]—> ((final))</code>
Unión (Alternativa)	<code>a b</code>	<code>(inicio) —ε—> [a] —ε—> ((final))</code> <code> └ ε —> [b] —ε —┘</code>
Cerradura de Kleene	<code>a*</code>	<code>(inicio) —ε—> [a] —ε—> ((final))</code> <code> </code> <code> └ ε —> [a] —ε —┘</code> <code> └ ε —> [a] —ε —┘</code>

3. Algoritmo de Construcción de Subconjuntos (De AFND a AFD)

Como las computadoras no pueden procesar transiciones múltiples simultáneas (ϵ), se aplica la construcción de subconjuntos:

- ϵ -clausura(s): Conjunto de todos los estados alcanzables desde s usando únicamente transiciones ϵ (incluyendo al propio s).
- $\text{mueva}(T, a)$: Conjunto de estados a los que se llega desde cualquier estado del conjunto T consumiendo el símbolo a .
- Transición en el AFD: $\delta_{\text{AFD}}(T, a) = \epsilon\text{-clausura}(\text{mueva}(T, a))$.

Ejemplo Resuelto de Parcial: Regex para Identificadores y Decimales

- **Identificador:** `[a-zA-Z_][a-zA-Z0-9_]*` (Inicia con letra o guion bajo, seguido de letras, dígitos o guiones bajos).
- **Número Decimal / Real:** `[0-9]+ ('.' [0-9]+)?` (Uno o más dígitos enteros, opcionalmente seguidos de un punto y más dígitos).

5. ANTLR 4: Gramáticas, Precedencia, Listener y Visitor

ANTLR 4 (ANother Tool for Language Recognition) es el generador de analizadores sintácticos más potente de la industria. Utiliza algoritmos ALL(*), resolviendo de forma nativa la precedencia de operadores y la recursión izquierda directa.

1. Estructura de una Gramática en ANTLR 4 (`ScientificCalc.g4`)

```
grammar ScientificCalc;

// ===== REGLAS DEL PARSER (Inician con MINÚSCULA) =====
prog: stat+ ;

stat: expr NEWLINE          # printExpr
    | ID '=' expr NEWLINE    # assign
    | 'clear' NEWLINE        # clearMem
    ;

// En ANTLR 4 la precedencia se define POR EL ORDEN DE LAS REGLAS (Arriba = Mayor prioridad)
expr: '(' expr ')'          # parens
    | '<assoc=right>' expr '^' expr # power      // Mayor precedencia: Potencia a la derecha
    | ('+'|'-') expr        # unarySign        // Signo unario
    | expr op=('*'|'/') expr # mulDiv          // Multiplicación y División
    | expr op=('+'|'-') expr # addSub          // Menor precedencia: Suma y Resta
    | function '(' expr ')'  # funcCall
    | ID                    # id
    | NUMBER                # num
    ;

function: 'sin' | 'cos' | 'tan' | 'sqrt' | 'log' | 'ln' | 'abs' ;

// ===== REGLAS DEL LEXER (Inician con MAYÚSCULA) =====
NUMBER : [0-9]+ ('.' [0-9]+)? ;
ID      : [a-zA-Z_][a-zA-Z0-9_]* ;
NEWLINE : '\r'? '\n' ;
WS       : [ \t]+ -> skip ; // La directiva skip descarta espacios y tabulaciones
```

2. Las 4 Reglas de Oro de ANTLR 4

1. Diferenciación Léxico / Sintáctico: Reglas con MAYÚSCULA son del Lexer (tokens); reglas con minúscula son del Parser.
2. Precedencia de Operadores: Las alternativas escritas más arriba en la regla `expr` tienen mayor prioridad matemática.
3. Asociatividad: Por defecto es a la izquierda ($1 - 2 - 3 = (1 - 2) - 3$). Para potencias se usa explícitamente `<assoc=right>` ($2 ^ 3 ^ 2 = 2 ^ (3 ^ 2) = 512$).
4. Etiquetas de Alternativa (`# etiqueta`): Generan métodos específicos en el Visitor (ej. `visitAddSub`, `visitPower`), permitiendo un código desacoplado y sin `if-else` gigantes.

3. Patrón Visitor vs Patrón Listener

Característica	Patrón Visitor (Recomendado para Evaluadores)	Patrón Listener (Recomendado para Traductores/Linters)
Control del Recorrido	Manual / Explícito. Tú decides cuándo y cómo llamar a <code>visit(nodoHijo)</code> .	Automático / Pasivo. Un <code>ParseTreeWalker</code> recorre todo el árbol en profundidad.
Retorno de Valores	Sí. Retorna tipos genéricos: <code>extends ScientificCalcBaseVisitor<Double></code> .	No. Todos los métodos retornan <code>void</code> (requiere pilas o mapas externos).
Evaluación Condicional / Bucles	Excelente. Puedes evaluar solo una rama de un <code>if</code> o evaluar 800 veces en <code>plot</code> .	Imposible. Siempre visita cada nodo del árbol exactamente una vez.
Firma de Métodos	<code>Double</code> <code>visitAddSub(ScientificCalcParser.AddSubContext ctx)</code>	<code>void enterAddSub(...)</code> y <code>void exitAddSub(...)</code>

6. Banco de Preguntas Típicas de Examen Parcial

Pregunta 1: ¿Cuál es la diferencia entre la fase de Análisis Sintáctico (Parser) y la de Análisis Semántico?

Respuesta: El Parser valida que la estructura y el orden de los tokens cumplan la gramática (por ejemplo, que no falte un paréntesis o un operador). El Análisis Semántico valida el significado lógico y la congruencia de los datos (por ejemplo, que una variable esté declarada antes de usarse, o que no se intente sumar un texto con un número booleano).

Pregunta 2: ¿Qué es una gramática ambigua y por qué representa un grave problema en compiladores?

Respuesta: Una gramática es ambigua si para una misma cadena de entrada existen dos o más árboles de análisis sintáctico válidos. Es un problema crítico porque un compilador no sabría qué significado semántico darle al código (ejemplo clásico: la expresión `2 + 3 * 4` podría evaluarse como `(2 + 3) * 4 = 20` o como `2 + (3 * 4) = 14`).

Pregunta 3: En ANTLR 4, ¿qué función cumple la directiva `-> skip` en las reglas del Lexer?

Respuesta: Indica al Lexer que, tras reconocer los caracteres coincidentes (como espacios en blanco `[ \t]+` o comentarios), los descarte completamente y no genere un token para el Parser, manteniendo limpia la gramática sintáctica.

Pregunta 4: En un evaluador basado en el patrón Visitor, ¿por qué es fundamental tener un `Map<String, Double> memory`?

Respuesta: Actúa como la Tabla de Símbolos en tiempo de ejecución. Permite persistir el valor de variables asignadas (ej. `a = 10`) y recuperarlas cuando se consultan en expresiones posteriores (ej. `a * 2`).

Pregunta 5: ¿Cómo logra el comando `plot(f(x), xmin, xmax)` graficar una función sin tener que recompilar la expresión matemática en cada punto?

Respuesta: Aprovecha que el árbol sintáctico (AST) generado por ANTLR es inmutable en memoria. Divide el intervalo en muestras (ej. 800 puntos) y en un bucle actualiza `memory.put("x", valorActual)` e invoca `visit(tree)`, reutilizando el mismo árbol para calcular cada punto (x, y) en microsegundos.

Pregunta 6: ¿Qué diferencia existe entre un análisis sintáctico Top-Down (LL) y uno Bottom-Up (LR)?

Respuesta: Top-Down (LL): Construye el árbol desde la raíz (símbolo inicial) hacia las hojas (tokens), prediciendo las producciones. Bottom-Up (LR): Comienza en las hojas (tokens de entrada) y realiza operaciones de *desplazamiento* (*shift*) y *reducción* (*reduce*) hasta alcanzar la raíz.

🚀 Resumen de Fórmulas y Reglas Mnemotécnicas para el Parcial

- Jerarquía: Regulares (AF) $\subset$ Libres de Contexto (Pila) $\subset$ Sensibles al Contexto $\subset$ Enumerables (Turing).
- Thompson: Concatenación = En serie | Unión = En paralelo con ε | Cerradura = Bucle con ε .
- Subconjuntos: `Nuevo Estado =  $\varepsilon$ -clausura(mueve(EstadoActual, símbolo))`.
- Precedencia en ANTLR: Lo más alto en la lista de alternativas se evalúa con mayor prioridad.
- Visitor: Control de flujo manual, tipo de retorno genérico `T`, ideal para calculadoras y lenguajes interpretados.